

Phase 2

FitBuddy- AI Fitness Plan Generator using Gemini Models

Requirement Analysis

1. Introduction

Requirement Analysis is the second major phase of the FitBuddy – AI Fitness Plan Generator project. After identifying the project concept during the Brainstorming and Ideation Phase, this phase focuses on identifying and organizing the requirements necessary to develop the application.

The purpose of requirement analysis is to clearly understand:

- What the user should be able to do.
- What the application should provide.
- What information the system needs from the user.
- How the AI component should process the information.
- How workout plans should be generated.
- How feedback should modify an existing plan.
- How user and plan information should be stored.
- How administrators or coaches should access stored information.
- What technical components are required to implement the system.

The FitBuddy project documentation describes the application as an AI-powered fitness planning system that generates personalized workout plans, provides nutrition or recovery tips, accepts feedback, updates plans, and maintains user and plan information.

2. Purpose of Requirement Analysis

The main purpose of this phase is to convert the idea developed in Phase 1 into a clear set of system requirements.

Requirement analysis provides a foundation for the later stages of:

- System design
- Application development
- Database implementation
- AI integration
- Frontend development
- Testing
- Deployment

A clear requirement specification helps ensure that the developers understand the expected behavior of the FitBuddy application before implementation begins.

3. Understanding the FitBuddy System

FitBuddy is designed as a web-based application where a user enters fitness-related information and receives a personalized workout plan.

The documented application accepts information including:

- Name
- User ID
- Age
- Weight
- Fitness Goal
- Workout Intensity

The source project describes the workout-generation process as producing a personalized 7-day workout schedule based on the user's fitness goal and intensity.

The application also provides nutrition or recovery guidance and supports feedback-based modification of generated plans.

4. Stakeholders and Users

Requirement analysis begins by identifying the people or components that interact with the system.

4.1 Regular User

The regular user is the main participant in the FitBuddy system.

The user should be able to:

- Enter personal information.
- Select a fitness goal.
- Select workout intensity.
- Generate a workout plan.
- View the generated plan.
- Read nutrition/recovery advice.
- Submit feedback.
- Receive an updated workout plan.

4.2 Administrator or Coach

The project documentation identifies an administrator or coach scenario.

An administrator or coach can access a view containing registered users and their workout plans.

The documented administrative view can display:

- Username
- Age
- Weight
- Fitness Goal
- Workout Intensity
- Original workout plan

- Updated workout plan

4.3 AI Service

The Generative AI service is another important system participant.

It receives appropriate information from the backend and produces:

- Workout plans
- Nutrition/recovery tips
- Revised workout plans

5. Figure 1 – Stakeholder Identification

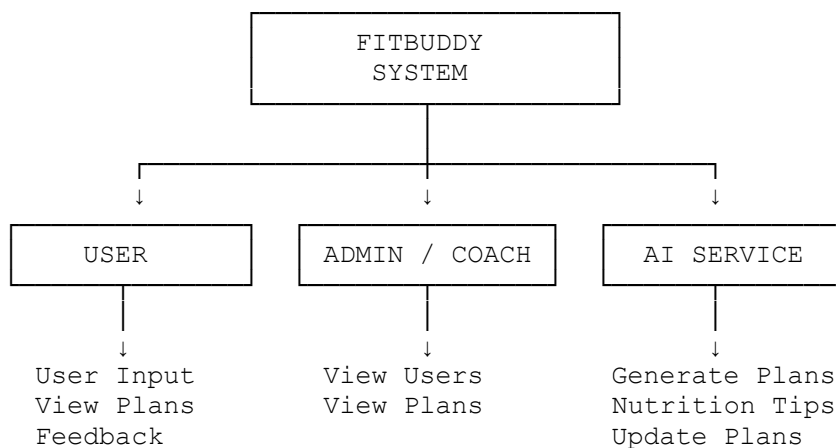


Figure 1: FitBuddy Stakeholder Identification

6. Functional Requirements

Functional requirements describe the actions and services that FitBuddy should perform.

FR-01: User Input Collection

The system should provide a form through which users can enter their fitness information.

The documented input fields include:

- Name
- User ID
- Age
- Weight
- Fitness Goal
- Workout Intensity

FR-02: Fitness Goal Selection

The system should allow users to specify their fitness objective.

The project documentation refers to goals such as:

- Weight loss
- Muscle gain
- General wellness

The selected goal becomes an important input for AI-generated recommendations.

FR-03: Workout Intensity Selection

The user should be able to select the desired workout intensity.

The documented interface uses:

- Low
- Medium
- High

This information is used to influence the generated workout plan.

FR-04: Personalized Workout Generation

The primary functional requirement is the ability to generate a personalized workout plan.

The project documentation describes the function:

```
generate_workout_gemini()
```

This function generates a structured 7-day workout plan using user inputs such as fitness goal and intensity.

FR-05: Day-Wise Workout Structure

The generated plan should be organized into a structured schedule.

The documented workout output can include:

- Warm-up
- Main workout
- Exercise details
- Sets
- Repetitions
- Cool-down

- Recovery guidance

This structure is intended to make the generated plan understandable and usable.

7. Nutrition and Recovery Requirements

FR-06: Nutrition Tip Generation

FitBuddy should provide a concise nutrition or recovery tip along with the workout plan.

The documented function is:

```
generate_nutrition_tip_with_flash()
```

It generates practical suggestions related to the user's fitness goal.

Examples described in the project documentation include:

- Hydration advice
- Protein suggestions
- General recovery practices

FR-07: Goal-Based Nutrition Guidance

The nutrition/recovery information should relate to the user's selected fitness goal.

For example, the project documentation describes nutrition advice that can complement a user's workout objectives.

8. Feedback Requirements

FR-08: Feedback Submission

The system should allow users to submit feedback regarding their current workout plan.

Feedback may contain requests such as:

- Add yoga.
- Include more cardio.
- Modify the difficulty.
- Include additional recovery time.

The source documentation specifically describes examples such as “Add yoga” and “Include more cardio.”

FR-09: Feedback-Based Plan Modification

The application should use submitted feedback to modify the original workout plan.

The documented function is:

```
update_workout_plan()
```

The function receives the original plan and user feedback and generates a revised plan.

FR-10: Preserve Original and Updated Plans

The system should retain the original plan while also storing the updated plan.

This allows the application to maintain a record of how the workout plan changed after feedback.

The documented implementation stores original and updated plans as separate information.

9. Database Requirements

FR-11: Store User Information

The system should store user information including:

- User ID
- Name
- Age
- Weight
- Fitness Goal
- Workout Intensity

The documented implementation uses SQLite and SQLAlchemy for persistence.

FR-12: Store Generated Plans

The system should store the AI-generated workout plans.

This allows the plans to be retrieved later and supports the feedback-based updating process.

FR-13: Update Stored Plans

When a user submits feedback, the updated plan should be stored.

The documented database functions include:

- `save_user()`
- `save_plan()`
- `update_plan()`

- `get_original_plan()`
- `get_user()`
- `get_all_users()`
- `get_all_plans()`

10. Administrative Requirements

FR-14: View All Users

The system should provide an administrative view of registered users.

The documented route is:

```
/view-all-users
```

The corresponding template is:

```
all_users.html
```

This view allows administrators to inspect user information and plans.

FR-15: View Original and Updated Plans

The administrative view should make it possible to compare the original workout plan with the updated plan.

This supports monitoring of plan changes resulting from user feedback.

11. Backend Requirements

FR-16: Request Processing

The FastAPI backend should receive user information from the frontend.

The documented implementation uses FastAPI form parameters and Pydantic structures such as:

- `UserInput`
- `FeedbackRequest`

FR-17: Route Management

The system should provide routes for the major operations.

The documented routes are:

Route	Purpose
/	Displays the main input form
/generate-workout	Generates the workout and nutrition information
/submit-feedback	Processes feedback and updates the plan
/view-all-users	Displays users and their plans

These routes form the main operational flow of the documented FitBuddy application.

12. AI Requirements

The AI layer is a central component of FitBuddy.

FR-18: Workout Generation

The AI service should generate a structured workout plan from relevant user information.

The documented function is:

```
generate_workout_gemini()
```

R-19: Nutrition Generation

The AI service should generate a concise nutrition or recovery tip.

The documented function is:

```
generate_nutrition_tip_with_flash()
```

FR-20: Workout Plan Revision

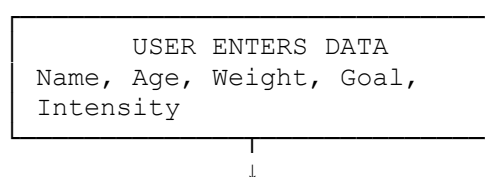
The AI service should modify an existing plan based on user feedback.

The documented function is:

```
update_workout_plan()
```

The three AI functions form the main Generative AI functionality described in the project report.

13. Figure 2 – Functional Requirement Flow



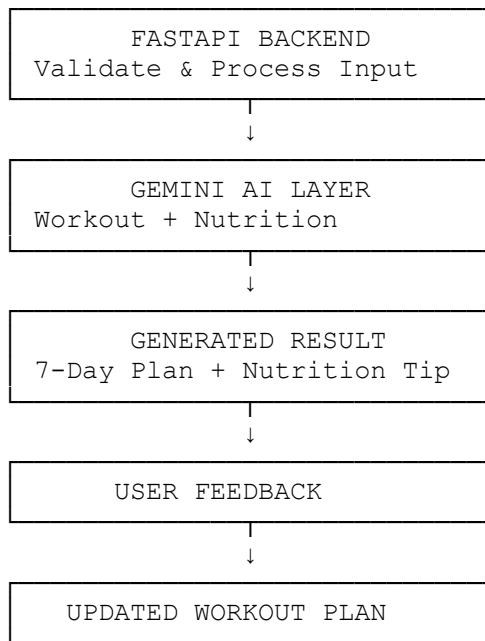


Figure 2: Functional Requirement Flow of FitBuddy

14. Non-Functional Requirements

Non-functional requirements describe the qualities and operational expectations of the system.

14.1 Usability

The application should provide an understandable user interface.

The project documentation describes a clean fitness-oriented interface where users can enter their information and generate their plans.

14.2 Responsiveness

The interface should work across different screen sizes.

The documented frontend uses Flexbox and media queries for responsive presentation.

14.3 Performance

AI requests should be processed efficiently so that users can receive generated information without unnecessary delay.

The documented architecture separates workout generation and nutrition-tip generation into appropriate AI functions.

14.4 Maintainability

The system should use modular components so that individual parts can be developed and modified independently.

The documented architecture separates:

- Frontend
- Backend
- AI services
- Database

This modular organization supports maintainability.

14.5 Reliability

The application should process valid user input consistently and return the appropriate output.

The database should preserve user and plan information so that generated plans can be retrieved and updated.

14.6 Security Considerations

Sensitive configuration information such as the Gemini API key should not be hard-coded into the application.

The project documentation specifies using an environment variable such as:

```
GOOGLE_API_KEY
```

and storing it in a `.env` file when appropriate.

15. Input Requirements

The main input requirements identified for FitBuddy are:

Input	Purpose
Name	Identifies the user
User ID	Identifies the user's record

Input	Purpose
Age	Provides user information for personalization
Weight	Provides fitness-related information
Fitness Goal	Determines the main objective
Workout Intensity	Helps determine workout difficulty
Feedback	Provides changes requested by the user

These inputs form the basis for the documented AI-generation and feedback workflow.

16. Output Requirements

The system should produce outputs including:

Output 1 – Personalized Workout Plan

A structured 7-day workout schedule.

Output 2 – Nutrition/Recovery Tip

A concise recommendation related to the selected fitness goal.

Output 3 – Updated Workout Plan

A revised plan based on user feedback.

Output 4 – Administrative User Information

A view containing user details and original/updated plans.

17. Figure 3 – Input and Output Relationship

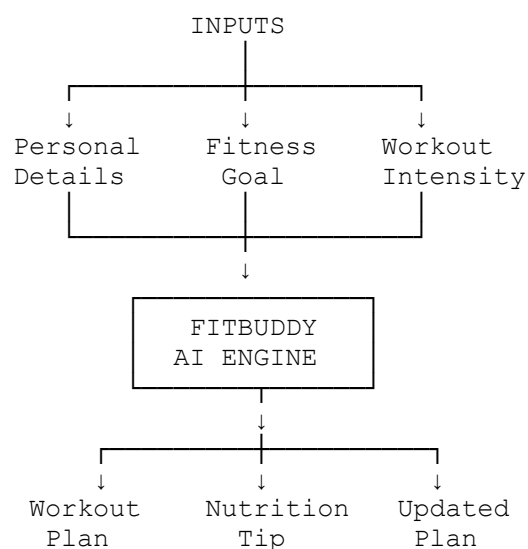


Figure 3: FitBuddy Input-Output Relationship

18. Technical Requirements

The documented FitBuddy implementation identifies the following technical requirements.

18.1 Programming Language

Python is required for the backend and AI integration.

18.2 Web Framework

FastAPI is used for backend routing and request processing.

18.3 AI Service

Google Gemini APIs are used for Generative AI functionality.

18.4 Database

SQLite is used for persistent storage.

18.5 ORM

SQLAlchemy is used to interact with the database.

18.6 Frontend

HTML and CSS are used to create the user interface.

18.7 Template Engine

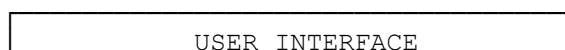
Jinja2 is used to dynamically render application pages.

18.8 Server

Uvicorn is used to run the FastAPI application locally.

These technologies are identified in the supplied project documentation

19. Figure 4 – Technical Requirement Layers



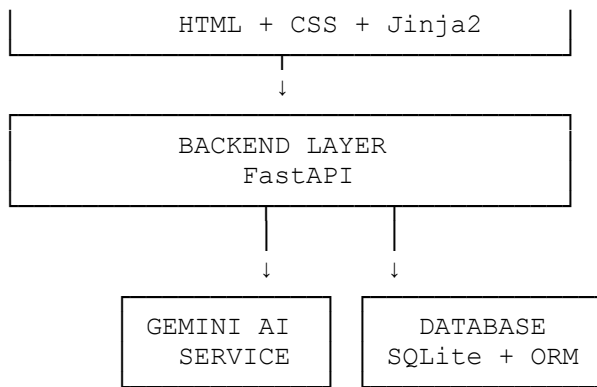


Figure 4: FitBuddy Technical Requirement Layers

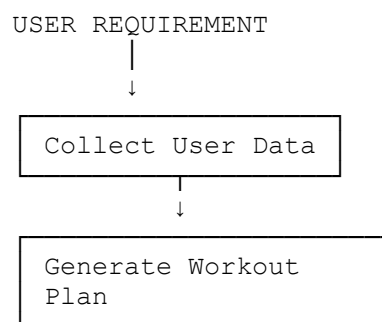
20. Requirement Traceability

Requirement traceability connects the identified requirements with the corresponding FitBuddy functionality.

Requirement	FitBuddy Feature
User information collection	Input form
Fitness goal	Goal selection
Workout intensity	Intensity selection
Workout generation	<code>generate_workout_gemini()</code>
Nutrition guidance	<code>generate_nutrition_tip_with_flash()</code>
Feedback	Feedback form
Plan modification	<code>update_workout_plan()</code>
Data persistence	SQLite + SQLAlchemy
User management	User database
Administrative monitoring	<code>/view-all-users</code>
Dynamic display	Jinja2 templates

The project documentation explicitly connects these functions and routes to the corresponding FitBuddy features.

21. Figure 5 – Requirement Traceability



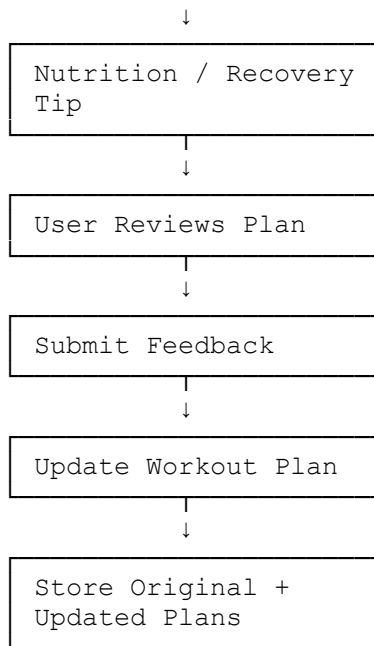


Figure 5: Requirement Traceability Flow

22. Requirement Prioritization

The requirements can be grouped according to their importance to the main application.

High-Priority Requirements

These are essential to the primary purpose of FitBuddy:

1. User input collection
2. Fitness goal selection
3. Workout intensity selection
4. AI workout generation
5. Display of generated workout plan
6. Nutrition/recovery tip generation
7. Database storage

Medium-Priority Requirements

These extend the core functionality:

1. User feedback
2. Feedback-based plan modification
3. Storage of updated plans
4. Administrative user view

Supporting Requirements

These support application quality:

1. Responsive interface
2. Modular architecture
3. Environment-variable configuration
4. Local API documentation
5. Maintainable project organization

23. Assumptions

The requirement analysis is based on the following assumptions documented or implied by the supplied project design:

- Users provide valid basic fitness information.
- The AI service is accessible when a plan is generated.
- The required API key is configured correctly.
- The database is available to the application.
- Users can access the application through a web browser.
- The application is primarily intended as a fitness-planning application.

The source document does not provide a complete clinical validation process or medical decision-support specification, so such capabilities should not be assumed to be part of the current project.

24. Constraints

The FitBuddy project has several practical constraints.

24.1 AI Dependency

Workout generation depends on the configured Generative AI service.

24.2 API Configuration

The application requires the appropriate API key and configuration.

24.3 Internet/API Availability

AI-powered requests require access to the configured service.

24.4 Input Quality

Generated results depend partly on the information supplied by the user.

24.5 Database Scope

The documented project uses SQLite, which is appropriate for the documented application but may require architectural changes for a larger production deployment.

25. Requirement Analysis Outcome

At the end of the Requirement Analysis Phase, the major functional and technical requirements of FitBuddy have been identified.

The system must be able to:

1. Collect user information.
2. Accept fitness goals.
3. Accept workout intensity.
4. Generate personalized workout plans.
5. Generate nutrition/recovery tips.
6. Accept user feedback.
7. Modify workout plans according to feedback.
8. Store user and plan information.
9. Preserve original and updated plans.
10. Display users and plans through the administrative view.
11. Provide a responsive web interface.
12. Connect the frontend, backend, AI services, and database.

26. Phase 2 Deliverables

The major deliverables of the Requirement Analysis Phase are:

1. Stakeholder Identification

Identification of users, administrators/coaches, and AI services.

2. Functional Requirements

Detailed identification of the actions FitBuddy must perform.

3. Non-Functional Requirements

Identification of usability, responsiveness, maintainability, reliability, performance, and security considerations.

4. Input Specification

Identification of information required from users.

5. Output Specification

Identification of workout plans, nutrition tips, updated plans, and administrative information.

6. Technical Requirements

Identification of Python, FastAPI, Gemini, SQLite, SQLAlchemy, HTML/CSS, Jinja2, and Uvicorn.

7. Requirement Traceability

Mapping requirements to specific FitBuddy features, functions, routes, and database behavior.

27. Transition to Phase 3

The requirements identified in Phase 2 provide the foundation for **Phase 3 – Project Design**.

During the design phase, these requirements can be transformed into:

- System architecture
- Database design
- Data flow
- AI integration design
- User interface design
- Backend structure
- Frontend structure
- Module relationships

The documented FitBuddy architecture already identifies four major technical layers:

Frontend → FastAPI Backend → AI Services / Database This architecture can be developed in greater detail during the Project Design Phase.

PHASE 2 CONCLUSION

The Requirement Analysis Phase converts the FitBuddy idea into a structured set of functional, technical, and quality requirements.

The most important requirement is the ability to accept user fitness information and generate a personalized workout plan using Generative AI. Supporting requirements include nutrition/recovery guidance, feedback-based plan modification, database persistence, and administrative monitoring.

The requirements identified in this phase provide a clear foundation for designing and developing the FitBuddy system.

